

APRENDIZAJE INCREMENTAL

- Con el aumento de las bases de datos, existe una necesidad constante de gestionar también el aumento de los datos de entrenamiento
 - Una posible solución a este problema es utilizar técnicas de aprendizaje incremental en las que los datos se procesan por lotes o *batches*
 - Se consideran subconjuntos de los datos en un momento dado
 - Luego se combina el resultado para ahorrar memoria

APRENDIZAJE INCREMENTAL

- Este enfoque de aprendizaje incremental se aplica también cuando las bases de datos se va incrementando gradualmente
 - Aprendizaje en línea (*on-line*)
 - Distinto del aprendizaje incremental
 - Cada nuevo subconjunto que se añade es un nuevo lote (*batch*) de entrenamiento
 - El modelo se va actualizando con cada lote
 - Aprender de forma incremental a partir de los resultados obtenidos con los lotes anteriores



APRENDIZAJE INCREMENTAL

- Es decir, el aprendizaje incremental puede utilizarse
 - para mantener el consumo de memoria y tiempo del algoritmo de aprendizaje en un nivel manejable
 - porque se necesita hacer predicciones en un momento en el que aún no se dispone de todos los datos
 - Similar al aprendizaje en línea o *Stream Learning*

APRENDIZAJE INCREMENTAL

- Una cuestión importante del aprendizaje incremental es si el concepto objetivo puede cambiar o si se supone que es constante
 - Si puede cambiar → deriva del concepto
 - *Concept drift*
 - Los ejemplos antiguos pueden inducir a error
 - Son ejemplos de un concepto objetivo antiguo que puede ser muy diferente del concepto que se intenta aprender
 - Más común en *Stream Learning*

APRENDIZAJE INCREMENTAL

- Una cuestión importante del aprendizaje incremental es si el concepto objetivo puede cambiar o si se supone que es constante
 - Si puede cambiar → deriva del concepto
 - Si no cambia
 - Todos los ejemplos contienen la misma información sobre el concepto objetivo
 - Se puede juzgar el rendimiento comparando sus resultados con los del algoritmo de aprendizaje entrenado con todos los datos simultáneamente

APRENDIZAJE INCREMENTAL

- Por tanto, este enfoque de aprendizaje incremental se puede aplicar en dos situaciones:
 - Bases de datos muy grandes
 - Datos divididos en lotes
 - Aprendizaje *on-line*
 - Lotes de datos nuevos añadidos continuamente
 - Ejemplo:
 - Datos bursátiles
 - ¿*Concept drift*?

APRENDIZAJE INCREMENTAL

- En general, la capacidad de aprender de forma incremental a partir de lotes (*batches*) de datos es una característica importante que hace que un algoritmo de aprendizaje sea más aplicable a los problemas del mundo real
 - Sin embargo, no todos los modelos lo soportan
 - RR.NN.AA. permiten el entrenamiento por lotes
 - Cada *epoch* (o pocos) con un lote distinto

APRENDIZAJE INCREMENTAL

- Las RR.NN.AA pueden ser entrenadas mediante lotes de forma natural
 - La topología del modelo no cambia
 - Un modelo con una configuración de pesos puede ser entrenado con un nuevo lote
 - Se hace tender los pesos a minimizar la función de *loss* para ese lote
 - Válido tanto para *concept drift* como para aprendizaje incremental
 - Muy habitual en *Deep Learning*
 - Bases de datos muy grandes
- Los SVM no pueden de forma “nativa”

INCREMENTAL SVM

- Las SVM son excelentes clasificadores con grandes cantidades de datos de alta dimensionalidad
 - Esto se debe a que la generalización de una SVM no depende de todos los datos de entrenamiento, sino sólo de un subconjunto
 - Vectores de soporte
 - Estos vectores de soporte “resumen” la información de ese espacio de datos
 - Reducción del conjunto de datos
 - Es decir, un SVM es capaz de resumir el espacio de datos de forma muy concisa

INCREMENTAL SVM

- Sin embargo, el entrenamiento de las SVM puede llevar mucho tiempo
 - Sobre todo cuando se trabaja con datos ruidosos
 - Calcular una SVM es muy costoso en términos de tiempo y consumo de memoria

INCREMENTAL SVM

- Sin embargo, el entrenamiento de las SVM puede llevar mucho tiempo
 - Solución: Dividir el *dataset* en lotes y entrenar de forma incremental
 - También válido para aprendizaje *on-line*
 - Sin embargo, un SVM entrenado no se puede actualizar con nuevos datos
 - Cada vez se entrena desde 0
 - ¿Cómo se puede entrenar con cada lote y mantener el conocimiento de los entrenamientos previos?

INCREMENTAL SVM

- Idea:
 - Utilizar la capacidad de SVM de “resumir” el espacio de datos
 - En forma de vectores de soporte
 - Este “resumen” se realizará para cada lote
 - Vendrá resumido por los vectores de soporte
 - Procesamiento secuencial
 - Antes de “resumir” un lote, a este se añaden los vectores de soporte que se tengan
 - Para que tenga toda la información previa, resumida
 - De esta manera, cada nuevo “resumen” contendrá toda la información previa

INCREMENTAL SVM

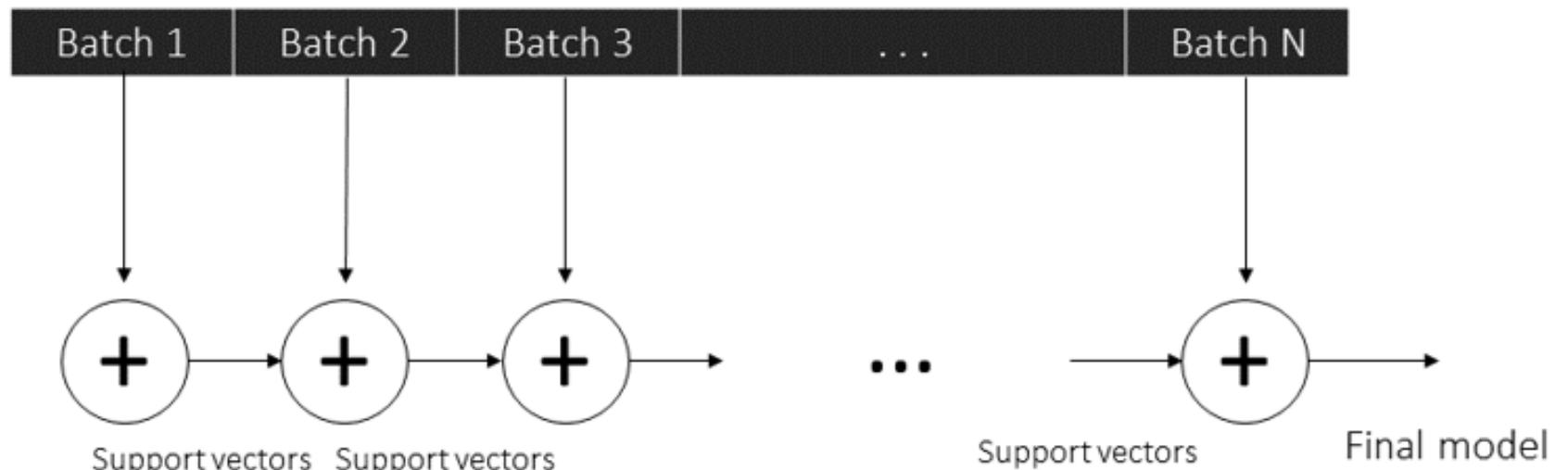
- Proceso:
 - Una vez se tiene un SVM entrenado con un subconjunto
 - Sus vectores de soporte resumen el entrenamiento que ha recibido
 - Ante un lote nuevo
 - Se combina este lote con los vectores de soporte que ya tiene
 - Se entrena otro SVM desde 0 con estos datos
 - Se repite este esquema para todos los lotes
 - Los vectores de soporte resumen el entrenamiento de todos los lotes previos

INCREMENTAL SVM

- Es decir, el entrenamiento incremental en un SVM funciona:
 1. Entrenar con un subconjunto de datos en el modelo
 2. Conservar sólo los vectores de soporte
 3. Añadirlos al siguiente subconjunto
 4. Volver al paso 1

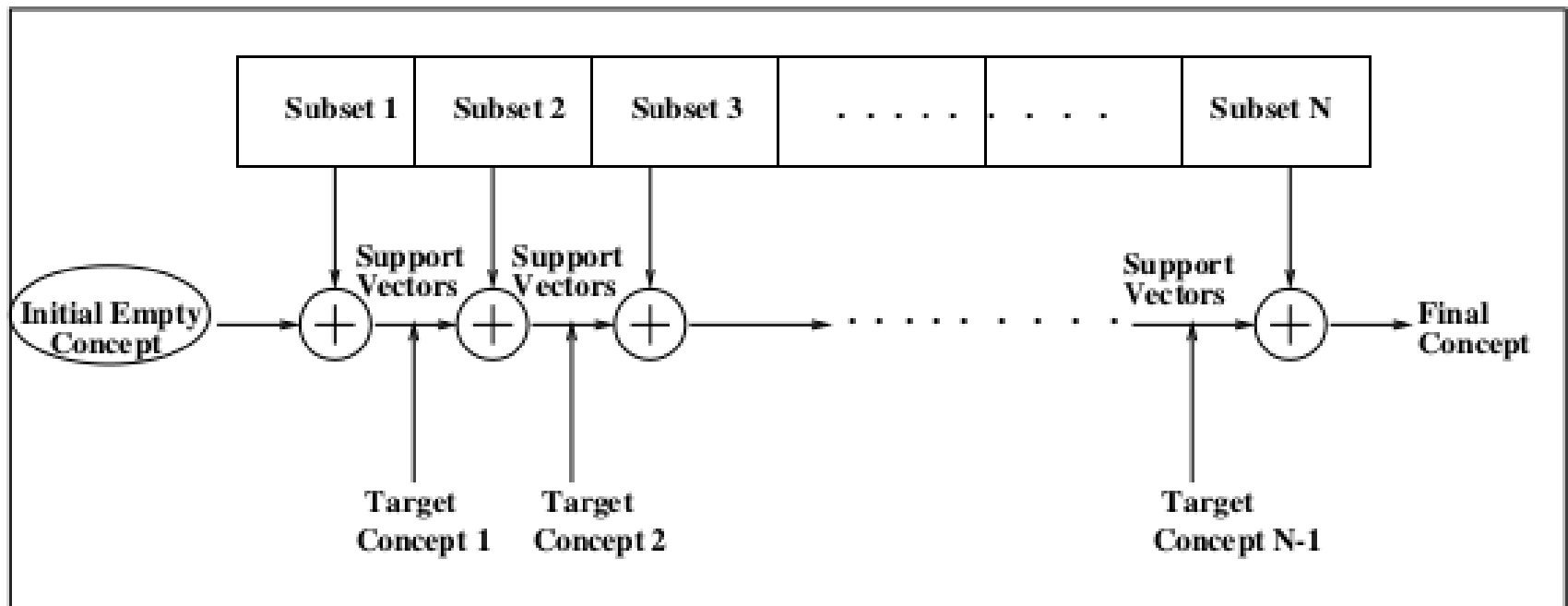
INCREMENTAL SVM

- Es decir, el entrenamiento incremental en un SVM funciona:



INCREMENTAL SVM

- Es decir, el entrenamiento incremental en un SVM funciona:



INCREMENTAL SVM

- Experimentos:
 - Se realizó un *10-fold crossvalidation*
 - Dividir el conjunto de datos en 10 subconjuntos y realizar 10 experimentos
 - Experimento i:
 - Conjunto de test (TE) → subconjunto i-ésimo
 - Conjunto de entrenamiento (TR) → resto de subconjuntos
 - En cada *fold*, los datos del conjunto de entrenamiento se dividen nuevamente en 10 subconjuntos disjuntos
 - TR1, TR2, ..., TR10
 - Para realizar aprendizaje incremental en ese *fold*
 - Y evaluar en el conjunto de test de ese *fold*

INCREMENTAL SVM

- Experimentos:
 - Se compararon dos casos:
 - Caso 1: Aprendizaje incremental visto
 - El modelo aprende a partir de nuevos datos y el resultado de los entrenamientos previos
 - Vectores de soporte
 - *Incremental Learning*
 - Caso 2: Se guardan todos los datos vistos anteriormente
 - Aprendizaje que utiliza todos los datos vistos hasta el momento
 - *Batch Learning*

INCREMENTAL SVM

- Experimentos:
 - Se llevan a cabo dos tipos de aprendizaje:
 - Tipo 1: Aprendizaje incremental. Para cada *fold*:
 1. Entrenar el SVM en TR1
 - Se hace test en TE
 2. Tomar los vect. de soporte SV1 de TR1 y añadirlos a TR2
 3. Entrenar con (SV1 + TR2)
 - Se hace test en TE
 4. Tomar SV2 de (SV1 + TR2) y añadirlos a TR3
 5. Entrenar con (SV2 + TR3)
 - Se hace test en TE
 6. Repetir este esquema hasta utilizar todos los TRi
 - Es decir, la última iteración se ejecutará en (SV(i-1) + TRi)
 - (SV9 + TR10)

INCREMENTAL SVM

- Experimentos:
 - Se llevan a cabo dos tipos de aprendizaje:
 - Tipo 1: Aprendizaje incremental
 - Tipo 2: Aprendizaje por lotes (*batch*). En cada *fold*:
 1. Entrenar el SVM utilizando TR1
 - Se hace test en TE
 2. TR2 se añade a TR1
 3. Entrenar con TR1+TR2
 - Se hace test en TE
 4. TR3 se añade a TR1 + TR2 y así sucesivamente hasta que todo el conjunto de entrenamiento original se utiliza para el entrenamiento

INCREMENTAL SVM

- Experimentos:
 - Se llevan a cabo dos tipos de aprendizaje:
 - Tipo 1: Aprendizaje incremental
 - Tipo 2: Aprendizaje por lotes (*batch*)
 - ¿Cuál de estos dos tipos va a dar un mejor *accuracy*?
 - ¿Por qué?

INCREMENTAL SVM

- Experimentos:
 - Se han utilizado distintos *datasets*
 - Muy conocidos por su uso como *benchmark*
 - Problemas de clasificación en dos clases

<i>Dataset</i>	<i># of Examples</i>	<i>No. of Attr.</i>
Australian	670	14
Diabetes	768	8
German	1000	24
Heart	270	13
Ionosphere	351	34
Liver-Disorders	345	6
Monks-1	432	6
Monks-2	432	6
Monks-3	432	6
Mushroom	8124	22
Promoter-Gene	106	57
Sonar	208	60

INCREMENTAL SVM

- Resultados:
 - *Accuracy* final de ambos métodos
 - Después de las 10 iteraciones

<i>Dataset</i>	<i>Batch method</i>	<i>Incr. method</i>
Australian	84.63	84.63
Diabetes	77.08	76.95
German	75.10	74.10
Heart	84.45	79.63
Ionosphere	94.57	93.14
Liver-Disorders	68.68	61.03
Monk-1	100	100
Monk-2	99.17	99.17
Monk-3	98.91	98.73
Mushroom	100	99.88
Promoter-Gene	93.55	92.55
Sonar	87.38	87.41
Average	88.87	87.27

INCREMENTAL SVM

- Resultados:
 - El *accuracy* del método *Batch Learning* es superior o igual a la del método incremental en 11/12 casos
 - Sin embargo la del método incremental es muy cercana a la del *Batch Learning*
 - Después de 10 iteraciones, el sistema incremental es capaz de recordar las partes relevantes de los datos
 - En consecuencia, la caída en *accuracy* de este modelo es insignificante o nula

INCREMENTAL SVM

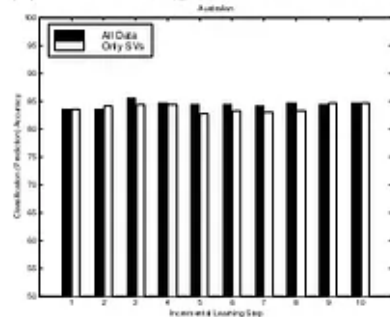
- Resultados:
 - La novedad de este enfoque es que, incluso tras sucesivos pasos de descarte de ejemplos redundantes, el conjunto de vectores de soporte recogido sigue siendo un conjunto representativo
 - ¿Cómo ha evolucionado el *accuracy* en cada paso incremental?

INCREMENTAL SVM

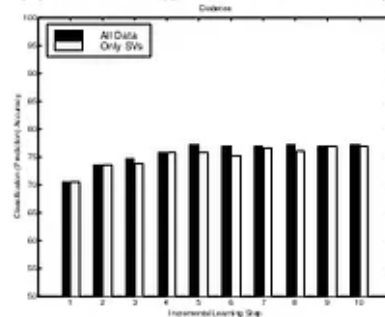
- Resultados:
 - Negro: *All Data*
 - Blanco: *Only SVs*
 - Eje X: paso incremental
 - 1-10
 - División de TR en 10 subconjuntos incrementales
 - Eje Y: *accuracy* de predicción del modelo obtenida al final del paso incremental correspondiente



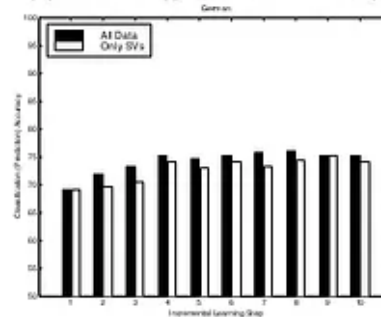
(1) Australian (gaussian $\sigma = 0.0005$)



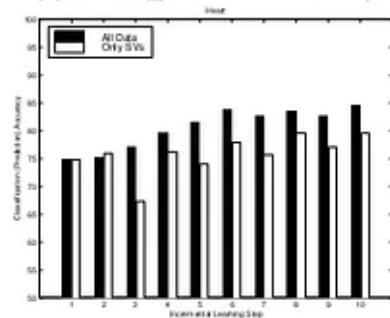
(2) Diabetes (gaussian $\sigma = 0.0005$)



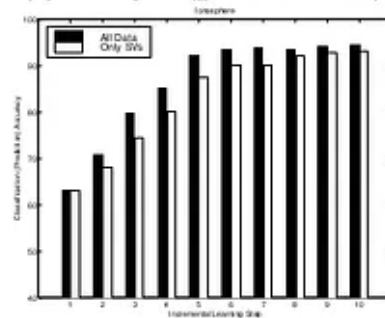
(3) German (gaussian $\sigma = 0.005$)



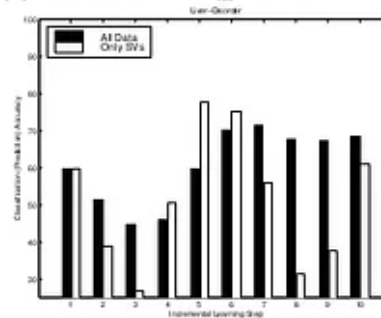
(4) Heart (gaussian $\sigma = 0.0005$)



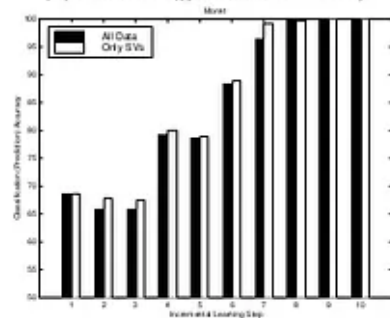
(5) Ionosphere (gaussian $\sigma = 1.6$)



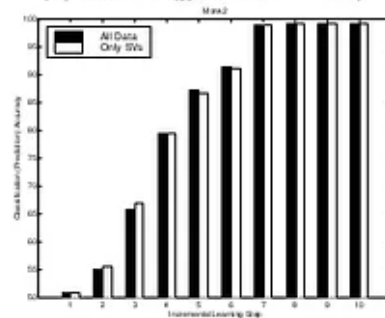
(6) Liver Disorders (gaussian $\sigma = 0.1$)



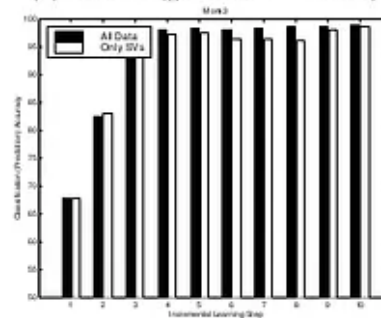
(7) Monk-1 (gaussian $\sigma = 0.1$)



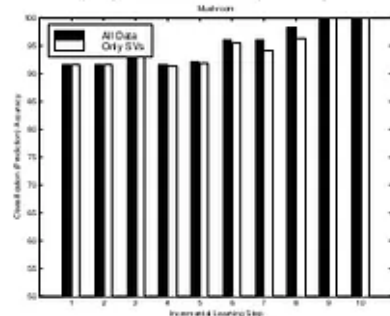
(8) Monk-2 (gaussian $\sigma = 0.1$)



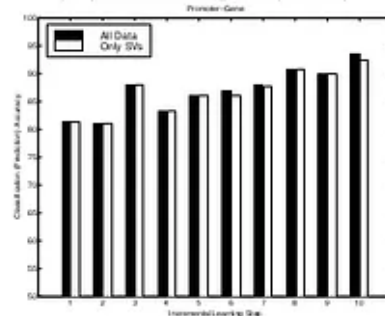
(9) Monk-3 (gaussian $\sigma = 0.001$)



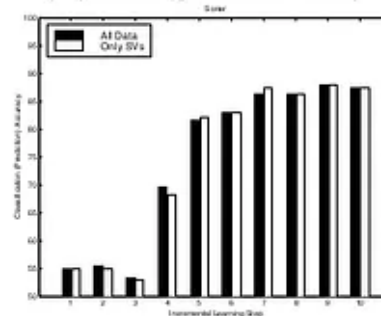
(10) Mushroom (linear)



(11) Promoter-gene (linear)



(12) Sonar (gaussian $\sigma = 0.4$)



INCREMENTAL SVM

- Experimento:
 - Objetivo: reducir la memoria necesaria para entrenar el modelo
 - Analizar si todos los vectores de soporte de nuestro conjunto son realmente necesarios
 - Intentar minimizar el conjunto lo máximo posible
 - Idea:
 - Entrenar un SVM y tomar los vectores de soporte
 - Entrenar de nuevo sólo con los vectores de soporte, pero eliminando algunos
 - Si todos son necesarios, ninguno será redundante
 - Entrenar el SVM con parte de los vectores de soporte dará lugar a un menor *accuracy*

INCREMENTAL SVM

- Experimento:
 - Procedimiento:
 - Entrenar un SVM sobre todo el conjunto de datos D
 - Como resultado, se obtiene el conjunto de vectores de soporte SV
 - Realizar un *10-fold crossvalidation* sobre SV
 - Se divide SV en 10 subconjuntos disjuntos
 - En cada fold
 - Conjunto de test → subconjunto SV_i
 - Conjunto de entrenamiento → resto de subconjuntos
 - Tiene un 90% de los vectores de soporte
 - Se toma la media de los *accuracies* en test

INCREMENTAL SVM

- Resultados:
 - Primera columna:
 - Porcentaje de ejemplos seleccionados como vectores de soporte
 - Segunda columna:
 - *Accuracy* medio en el *10-fold* sobre todo el *dataset*
 - *10-fold* sobre D, todo el *dataset*
 - Tercera columna:
 - *Accuracy* medio en este experimento
 - *10-fold* sobre SV, el “resumen” del *dataset*
 - Resultado de entrenar un SVM con todo el *dataset* D

INCREMENTAL SVM

- Resultados:

<i>Dataset</i>	<i>% of Exs. Selected</i>	<i>%Accuracy (All data)</i>	<i>%Accuracy (Selected Exs.)</i>
Australian	33.76	84.63	48.40
Diabetes	58.03	77.08	51.68
German	54.16	75.10	45.35
Heart	35.16	84.45	33.31
Ion.	53.05	94.57	84.77
Liver.	67.64	68.68	21.25
Monk-1	82.61	100.0	100
Monk-2	78.76	99.17	98.75
Monk-3	23.54	98.91	94.04
Mush.	05.99	100.0	100
Gene	69.19	93.55	79.46
Sonar	67.38	87.38	58.81
Avg.	52.44	88.87	67.99

INCREMENTAL SVM

- Resultados:
 - Eliminar incluso el 10% de los vectores de soporte reduce significativamente el *accuracy* del modelo
 - Esto demuestra que:
 - El conjunto ya es mínimo
 - Si se eliminan más vectores de soporte se perderá información vital sobre la distribución de clases
 - Las SVM son muy buenas para resumir el espacio de datos original en una forma compacta

INCREMENTAL SVM

- Referencia:
 - Nadeem Ahmed Syed, Huan Liu, Kah Kay Sung, Incremental Learning with Support Vector Machines, 1999